DataBase Design

Pamela Afful

Question 1 - Relational Database Design

a) Entity Relationship Diagram

b)

- Research (<u>Research ID</u> , AUTHOR ID, Subject, Title, Publication Date, Research Type)
- Authors (<u>Author ID</u> , Author Full Name, Title, Faculty, RESEARCH ID, DEPT ID)
- Downloads (<u>Download ID</u>, USER ID, RESEARCH ID, Download Date, Download Fee)
- Registered Users (<u>UserId</u>, DOWNLOAD ID, Name, Surname)
- Faculty (<u>DeptID</u>, Department, Faculty Name, AUTHOR ID)

c) Author ID->Author Name

d) A functional dependency determines how the value of one (or more) attribute in a table or row is identified by knowing the value of another (key) attribute- known as the determinant.

e) RESEARCH ID-> AUTHOR ID ->Department

f) RESEARCH ID -> Title

g) Research has a many-to-many relation to Registered Users: multiple Research IDs can be downloaded by multiple users and vice versa. The identification key in this case is composite as (Research ID, User ID), as this combination can uniquely identify each download.

The relation is in 3NF, as every non-prime attribute of the Downloads, Download Date, and Download Fee tables can be uniquely identified by the (Research ID, User ID). Additionally, each non-prime download attribute can also be uniquely linked to via the Download ID primary key in the Downloads table.

Question 2- Data Warehouses

a)

- Downloads Fact Table can be a central table that can be used to track multiple KPIs, including user traffic, number of downloads, total fees paid, etc. These are numerical and quantifiable.
- Research Fact Table can also be used to track additional KPIs, such as the number of papers published in a given year, authors of interest that are related to the university, or a specific department. This can prove especially important if the main objective is to track faculty and departmental research output and interest.

b)

- Downloads (Download ID, USER ID, RESEARCH ID, Download Fees, Download Date)
- Research (Research ID, Titles, AUTHOR ID, Publication Date, Publication Type, ISBN)

c) UCT Faculty: one-dimensional table that is slow changing.

d) Faculty ->Department. Faculties house multiple departments, which in this case form a hierarchy

```
SELECT Subject,ReasearchType,
COUNT(ResearchID)
FROM Fact_Research
GROUP BY CUBE (Subject,ReasearchType)
```

The above gives the number of Research IDs per Subject-ResearchType combination as well as the number of Research IDs across Subjects alone and across Research Types (e.g book, paper, theses) as well.

f) The Research Type rows would disappear as this detail would be "rolled up" into the Subject totals, which form the hierarchy in this example.*comeback

Question 3- NoSQL

a) I would make use of a relational database to store easily definable and structured relationships between entities. An example of where this would be a table that relates authors to the various titles they published at various dates, their faculty or department, etc.

b) The key-value store should be a simple lookup dictionary where the key instantly returns a value stored for that key efficiently. In this case, I would use a key-value model to store AuthorID: Subject_Information. That is, each author id pulls up the subjects or field of expertise of that particular author. This is quite advantageous due to the fast-growing popularity and usage of the research platform- users need to be able to access information as swiftly as possible.

c) I would use a document database to store the actual content of all the research ouputs; The research ID would be the document ID in this case. These databases are best for semi-structured data and, as such, would suit the various types of formats that research outputs can take. Since key attributes of the research IDs (title, author, subject, etc.) are stored in BSONs (allowing related information to be nested together), users can efficiently interact with specific parts of the text they want- without having to retrieve entire documents or use multiple joins.

d) I would use a column-oriented database to store Research ID interactions with respect to downloads, citations etc. make it easy to track and evaluate user analytics.

e) Authors and their collaborators that they have worked or cited in works- can be useful for tracking research similarity.

Question 4

a) Inputs : Users, Payments, Date/month

Outputs: Total Pyment per user

Mappers- will take each user/date combination as a key and output a payment value <user-date, payment>

Then each reducer will take the each user-date key and its associated value the one reducer that will handle the summing of the payments for that pair.

b) Assuming the data is partitioned daily:

Inputs:

- The data will be split across mappers in the form ((year, user) payment), <year, user> are the keys and payment is the value being mapped to.
- This will be distributed across mappers:

  1. Each mapper counts the payments per user for each month per year <month,year,user> and get the valu,e which in this case are the payments. For example if User 1 made 3 payments in 2024 (Jan-5 ,Jan-10 and July-10), 2 in (March-10, April -25) in 2025 , the corresponding outputs from the different mappers will be

- `Mapper 1 output \<(Jan,2024,user1)\> 15`
- `Mapper 2 output \<(Jul, 2024, user1)\> 10`
- `Mapper 2 output \<(Mar,2025, user1)\>10`
- `Mapper 2 output \<Apr, 2025, user1\> 25`

  2. These map outputs, for the different key combinations, are sent are then sent to the requisite reducers (in two stages) that sum up the payments per user per year. In this case, each reducer may sum up the payments for each user across years. Using the same Users example:

     – Reducer 1 output < (User1,2024)>= 25

     – Reducer 2 output <User1, 2025>=35

     – Across all users per year 2024, and 2025

  3. The final ouput report will be a the sum of payments for each user per year for 2024 and 2025

Table 1: Final Output Table

User ID	Year	Total Payment
User1	2024	25
User1	2025	35
User n	2024	xx
User n	2025	xx